



ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ, ಬೆಲಗಾವಿ
VISVESVARAYA TECHNOLOGICAL UNIVERSITY - BELAGAVI



GOVERNMENT OF KARNATAKA
DEPARTMENT OF COLLEGIATE AND TECHNICAL EDUCATION
GOVERNMENT ENGINEERING COLLEGE NARGUND-582207



Major Project

Report On

“HOME SECURITY SYSTEM USING ESP32-CAM”

Submitted in partial fulfillment of the requirement for the degree of

Bachelor of Engineering

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SUBMITTED BY:

1. AMIT DHAVALESHWAR	2NR22AD002
2. ANIL KUMAR R	2NR22AD004
3. HEMA CHALESARA	2NR22AD009
4. VISHNU R SHETTY	2NR22AD039

Under The Guidance of
Prof. Ashwini S.Kale

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
GOVERNMENT ENGINEERING COLLEGE NARGUND-582207

[2025-26]



GOVERNMENT OF KARNATAKA
DEPARTMENT OF COLLEGIATE AND TECHNICAL EDUCATION
GOVERNMENT ENGINEERING COLLEGE NARGUND-582207
(Affiliated To Visvesvaraya Technological University)
DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE 2025-26

Certificate

Certify that the project work entitled “**Home Security System Using ESP32-CAM**” carried out by, **Amit Dhavaleshwar (2NR22AD002), Anil Kumar R (2NR22AD004), Hema Chalesara (2NR22AD009), Vishnu R Shetty (2NR22AD039)** are bonafide students of 7th semester in partial fulfillment for the award of **Bachelor of Engineering in Artificial Intelligence and Data Science of The Visvesvaraya Technological University, Belagavi** during the year 2025-26. It is certified that all corrections or suggestions indicated for internal assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the aid degree.

Signature of the
Guide

Signature of the
Project Coordinator

Signature of the
HOD

Signature of the
Principal

External Viva

Name of the Examiners

Signature with date

1) _____
2) _____

DECLARATION

This is to declare that the Dissertation work entitled "**Home Security System Using ESP32-CAM**" is a bonafide work carried out by us at Government Engineering College, NARGUND in partial fulfillment of the requirements for the award of the degree of BACHELOR OF ENGINEERING in ARTIFICIAL INTELLIGENCE and DATA SCIENCE of VISVESWARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI under the guidance of **Prof. Ashwini S.Kale** Further it is declared to the best of our knowledge the work reported here in, does not form part of any other thesis or dissertation on the basis of which any other candidate was conferred a degree or award on earlier occasion.

Place: Nargund

Date:

AMIT DHAVALESHWAR	2NR22AD002
ANIL KUMAR R	2NR22AD004
HEMA CHALESARA	2NR22AD009
VISHNU R SHETTY	2NR22AD039

ACKNOWLEDGEMENT

We consider it a privilege whole heartedly to express our gratitude and respect to each and every one who guided and helped us in successful completion of this Major Project. We are grateful to our institution Government Engineering College Nargund for having provided us with facilities which has made this project a success.

We indebted to our beloved Guide “**Prof. Ashwini S. Kale** ” who was a constant source of enthusiasm and whose sincere guidance valuable suggestions and benevolent direction were mainly responsible for my successful completion of the project.

We indebted to our beloved Coordinator “**Prof. Deepa Manavachari**” who was a constant source of enthusiasm and whose sincere guidance valuable suggestions and benevolent direction were mainly responsible for my successful completion of the project.

We express whole hearted gratitude to “**Prof. Ashwini S. Kale**” who is our respectable Head of Dept. of Artificial Intelligence and Data Science. . We wish to acknowledge her help who made our task easy by providing with her valuable help and encouragement.

Our due thanks to “**Prof. Siddanagouda Patil** ” the principal, as we consider ourselves very lucky to have such excellent computing facilities and their inspiration throughout our professional course.

We also thank the non-teaching staff of Artificial Intelligence and Data Science. Who guided at the time of difficulties. Finally, we thank all those who are involved directly and indirectly in completion of our project.

AMIT DHAVALSHWAR	2NR22AD002
ANIL KUMAR R	2NR22AD004
HEMA CHALESARA	2NR22AD009
VISHNU R SHETTY	2NR22AD039

ABSTRACT

Advanced IoT-based Home Security and Surveillance System designed using the ESP32-CAM microcontroller, integrating artificial intelligence, real-time monitoring, and smart home automation principles. The system functions as a comprehensive, low-cost, and scalable security platform capable of detecting intrusions, capturing visual evidence, and alerting homeowners instantly through network-enabled communication.

The proposed model incorporates a combination of PIR motion sensors, ESP32-CAM-based image acquisition, Wi-Fi connectivity, and optional facial recognition modules to provide an intelligent, automated security workflow. Upon detecting human presence, the system activates image/video capture, processes the data, and transmits alerts to the user through a mobile device or web interface. This ensures rapid threat detection while minimizing false alarms through optimized sensor-camera coordination.

The platform adopts a modular architecture that supports future enhancements, including cloud storage, advanced computer vision models, and seamless integration with existing smart home devices. A user-friendly control interface enables homeowners to manage, monitor, and customize system operations remotely, ensuring both security and convenience.

Furthermore, the system demonstrates how IoT-enabled embedded platforms can bridge the gap between traditional security practices and modern intelligent surveillance requirements. By combining autonomous sensing, real-time data transmission, and adaptive monitoring, the ESP32-CAM-based security framework establishes a reliable foundation for next-generation smart home ecosystems. The project highlights not only the technical feasibility of low-power, networked security devices but also their potential to provide scalable, user-centric, and future-ready protection solutions suitable for a wide range of residential and commercial environments..

Table Of Contents

Acknowledgement.....	i	
Abstract.....	ii	
Table of contents.....	iii	
List of Figures.....	iv	
CHAPTER 1. INTRODUCTION		
1.1 Overview.....	1	
1.2 History of IOT Based Security System.....	2	
1.3 Applications.....	3	
1.4 Advantages And Disadvantages.....	4	
1.4.1 Advantages.....	4	
1.4.2 Disadvantages.....	5	
CHAPTER 2. PROBLEM STATEMENT AND OBJECTIVE		
2.1 Problem Statement.....	6	
2.2 Objective.....	7	
2.3 Motivation Of The Project.....	8	
CHAPTER 3. LITERATURE SURVEY.....		14
CHAPTER 4. SYSTEM REQUIREMENT SPECIFICATION		
4.1 Functional Requirements.....	15	
4.2 Non-Functional Requirements.....	16	
4.3 Hardware Requirements	17	
4.4 Software Requirements	21	
4.4.1 Creation And Installation Of Telegram Bot.....	22	
CHAPTER 5. METHODOLOGY		
5.1 Block Diagram	24	
5.2 System Architecture	27	
CHAPTER 6. IMPLEMENTATION		
6.1 Data Description	28	
6.2 Core System Operations.....	29	
6.3 Project Structure.....	32	
6.4 ESP32-CAM Image Acquisition and Validation Module.....	37	
6.5 ESP32-CAM Web Server Code and Functional Description..	41	
6.6 Key Implementation Features.....	43	

CHAPTER 7. MODEL DEMONSTRATION

7.1 Bluetooth Scanning	46
7.2 HC-05 Module Connection Interface.....	47

CHAPTER 8. RESULTS AND DISCUSSION

8.1 Telegram Alert Showing Real-time Face Detection.....	48
8.2 Servo Motor-Based Door Lock Operation using.....	49
Arduino Bluetooth Control	

CHAPTER 9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion	50
9.2 Future Scope	51

References	54
------------------	----

Appendix	56
----------------	----

List of figures

Figure 4.1: ESP32 CAM Module.....	17
Figure 4.2: Arduino Uno Board – Pin Configuration and Major Components...	18
Figure 4.3: Solderless Breadboard for Circuit Prototyping.....	19
Figure 4.4: Micro Servo Motor (Tower Pro SG90)	20
Figure 5.1: Work Flow Of AI-Based Home Security Using..... ESP32-CAM and Arduino Uno	23
Figure 5.2: Arduino IDE Software Flow for Face Detection..... and Access Control	25
Figure 7.10: Prototype Model – External View of Home..... Security System Using ESP32-CAM	44
Figure 7.2: Internal Circuit Connections Component Integration..... of Home Security System	45
Figure 7.3: Smartphone Interface Bluetooth Module Pairing	46
Figure 7.4: HC-05 Bluetooth Module Pair..... and Switch-Based Control	47
Figure 8.1: Snapshots Telegram Alert Showing Real-Time..... Face Detection	48
Figure 8.2: Servo Motor-Based Door Lock Operation Using	49
Arduino Bluetooth Control	

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In today's rapidly evolving digital world, ensuring the security of residential spaces has become a critical necessity. With the increasing rate of home intrusions, thefts, and unauthorized access attempts, conventional security systems often dependent on manual monitoring or basic alarm mechanisms are no longer sufficient to provide reliable protection. These traditional systems lack intelligence, adaptability, and real-time decision-making capabilities, resulting in delayed responses, false alarms, and limited owner control. The emergence of Internet of Things (IoT) technologies has revolutionized modern security solutions by enabling smart, interconnected devices capable of sensing, processing, and responding autonomously.

Project aims to design and implement an AI-assisted Home Security System using ESP32-CAM, integrating hardware sensing technologies with software-driven automation to deliver a comprehensive security framework. The system combines motion detection, image capturing, remote surveillance, and real-time alerts, enabling homeowners to monitor their premises from anywhere using internet-enabled devices. Unlike conventional systems that automatically grant access upon detection, this platform ensures owner verification, significantly enhancing security and reducing unauthorized entry risks. Rapid advancements in computer vision, IoT connectivity, and embedded systems have enabled the creation of smart home environments where multiple devices cooperate to ensure safety, speed, and user convenience

In addition, the system integrates modern communication features that enhance user control and awareness. Real-time alerts are sent directly to the homeowner, allowing quick evaluation of any detected activity. The user can then manually operate the door mechanism through a Bluetooth-based mobile application. This combination of automation and human decision-making provides a highly secure and flexible home security solution.

1.2 HISTORY OF IOT-BASED HOME SECURITY SYSTEMS

The evolution of home security systems has progressed significantly over the past few decades, moving from simple mechanical solutions to intelligent, AI-enabled platforms. Early home security methods primarily relied on mechanical locks, physical barriers, and human guards, which provided limited protection and required constant manual supervision. During the late 20th century, electronic security systems emerged, introducing burglar alarms, magnetic door sensors, and closed-circuit television (CCTV) cameras. Although these systems improved surveillance, they were mostly stand-alone and reactive, offering no remote monitoring or intelligent decision-making capabilities.

With the advancement of microcontrollers and embedded systems, home security solutions became more automated. Systems using controllers such as Arduino enabled sensor-based intrusion detection, automated alarms, and basic access control. However, these systems still lacked connectivity and depended on local alerts such as buzzers or sirens. The introduction of the Internet of Things (IoT) marked a major breakthrough, allowing security devices to connect to the internet and communicate with users in real time. IoT-based systems enabled features like remote monitoring, cloud data storage, and mobile notifications, significantly improving user awareness and response time.

The integration of wireless communication technologies such as Wi-Fi and Bluetooth further enhanced flexibility and ease of deployment. Smart cameras like the ESP32-CAM made visual surveillance affordable and compact, enabling real-time image capture and transmission over the internet. This evolution reflects a shift from passive security mechanisms to smart, AI-assisted IoT-based home security systems that offer enhanced safety, automation, and user-centric control.

Today, IoT-based home security systems continue to evolve with the adoption of cloud computing, machine learning, and edge AI, enabling faster processing and improved privacy. Integration with platforms such as mobile apps and messaging services provides seamless user interaction and instant notifications. As a result, modern home security solutions are becoming more intelligent, scalable, and reliable, meeting the growing demand for smart and connected living environments.

1.3 APPLICATIONS

1 Residential Security:

The system provides continuous monitoring of the home environment using motion detection and camera-based verification. It sends real-time alerts to homeowners, enabling quick responses during suspicious activities. Captured images or videos help confirm intrusions instantly. This greatly enhances the safety and protection of family and property.

2 Commercial / Shop Surveillance:

Shops and small businesses can use the system for round-the-clock surveillance. It detects unauthorized entry after business hours and notifies the owner immediately. The ESP32-CAM captures visual evidence for verification and record-keeping. This reduces theft, improves safety, and supports better shop management.

3 Remote Property Monitoring:

Owners can monitor distant locations such as farms, warehouses, or rental properties in real time. The system sends images, alerts, or live streams through the internet for remote visibility. This ensures continuous awareness even when the owner is far away.

4 Smart Home Integration:

The system can be connected with other IoT devices like alarms, smart locks, or lights. It enables automated, intelligent responses during security events. Users can control or configure the system from their smartphone or home network. This makes it a valuable component of modern smart home ecosystems.

5 Elderly and Child Safety Monitoring:

The system can be used to monitor the movement and safety of elderly people and children within the home. Real-time alerts and camera verification help caregivers ensure their well-being and respond quickly to emergencies.

1.4 ADVANTAGES AND DISADVANTAGES

1.4.1 ADVANTAGES

1.Low Cost and Affordable: The ESP32-CAM is significantly cheaper than traditional CCTV systems while still offering essential surveillance features. This makes it suitable for low-budget home security projects and student prototypes. Users can implement advanced monitoring without investing in expensive hardware.

2.Real-Time Monitoring: The system enables live video streaming and instant image capture of the home environment. Homeowners can view activities remotely using a smartphone or computer. This ensures continuous awareness of what is happening inside or outside the house. Real-time access helps in quick decision-making during security events.

3.Wireless Connectivity: Built-in Wi-Fi allows the ESP32-CAM to connect directly to the internet without extra modules. This simplifies circuit design and reduces wiring complexity. Wireless communication enables remote access from anywhere in the world

4.Instant Alerts: Whenever motion or unauthorized activity is detected, the system sends immediate notifications to the user. Alerts can include images or video clips for quick verification. This reduces response time and improves security effectiveness. Instant alerts help prevent theft or damage by enabling rapid action.

5.Compact Size: The small and lightweight design of the ESP32-CAM allows installation in hidden or space-limited areas. It does not disturb the interior design of the home. The compact nature makes it suitable for indoor and outdoor applications. Easy placement increases surveillance flexibility.

6.Camera-Based Verification: Visual evidence provided by captured images or videos helps confirm actual intrusions. This minimizes false alarms caused by pets or environmental changes. Users can clearly see the situation before taking action. It improves trust and reliability of the security

1.4.2 DISADVANTAGES

1. Internet Dependency: The system relies heavily on stable Wi-Fi or internet connectivity. Poor or unstable networks can delay alerts or affect live streaming. This may reduce performance during critical moments. Therefore, the effectiveness depends on network availability.

2 Limited Night Vision Capability: The ESP32-CAM requires additional IR LEDs for proper low- light performance. Without them, image clarity may decrease at night. This can affect intrusion detection in dark environments.

3 Processing Limitations: The ESP32-CAM cannot run heavy AI or advanced detection algorithms locally. Complex tasks require cloud services or an external processor. This limits its ability to perform detailed analysis on-device. Therefore, expansion may require additional computational components.

4 Sensor and Coverage Constraints: PIR sensors have limited detection angles and distances. Incorrect placement can reduce accuracy or miss certain areas. Environmental factors may influence sensor sensitivity. Thus, proper setup and alignment are crucial for optimal performance.

CHAPTER 2

PROBLEM STATEMENT AND OBJECTIVES

2.1 PROBLEM STATEMENT

Traditional home security systems rely on basic alarms or manual monitoring, which often results in delayed responses, false alerts, and limited user control. Many smart systems lack proper owner verification and can unintentionally grant access, creating security risks. There is a need for a low-cost, intelligent, and remotely accessible IoT-based solution that can accurately detect intrusions, capture real-time evidence, and immediately notify homeowners. An ESP32-CAM-based system can address these limitations by combining motion detection, image acquisition, and wireless communication to provide reliable and efficient home surveillance.

2.2 OBJECTIVES

1. IOT-based Security System with ESP32-CAM:

The proposed system focuses on developing an IoT-enabled home security solution using the ESP32-CAM module. It detects intrusions and automatically captures images or videos when suspicious activity is identified. This ensures real-time visual evidence, improving the reliability and effectiveness of home security monitoring.

2 . Intelligent Monitoring with Sensors and Camera:

The system integrates PIR motion sensors with an onboard camera and wireless communication modules to create an intelligent monitoring framework. Motion detection triggers the camera only when required, reducing false alarms caused by unnecessary movements. This approach enhances detection accuracy while conserving power and bandwidth.

3. Real-Time Remote Surveillance :

Real-time remote surveillance is achieved by transmitting alerts, captured images, and live video streams to the homeowner through Wi-Fi or cloud-based platforms. This allows users to monitor their homes from any location using a smartphone or computer.

4. Manual Verification for Better Control:

Instead of relying solely on automated access control systems, the design allows homeowners to manually verify intrusion alerts. Users can view the captured images or live feed before taking action, reducing the risk of false responses. This approach provides greater control and improves overall

security decision-making.

5. Scalable and Cost-Effective Design :

The system is designed to be affordable and scalable, making it suitable for widespread adoption. Additional features such as facial recognition, cloud storage, smart locks, and advanced computer vision algorithms can be integrated in the future. This flexibility ensures long-term usability and adaptability to evolving security needs.

6. Secure Data Transmission and Privacy Protection:

The system ensures secure transmission of images, videos, and alert data using encrypted Wi-Fi communication protocols. This protects sensitive visual information from unauthorized access or interception. By prioritizing data security and user privacy, the system builds trust and reliability in real-world applications.

7. Low Power Consumption and Efficient Resource Management :

The ESP32-CAM is optimized to operate with low power consumption by activating the camera and sensors only when motion is detected. Efficient resource management extends device lifespan and reduces energy costs. This makes the system suitable for continuous operation in residential environments.

8. Reliable Alert and Notification System:

The security system provides instant alerts through mobile notifications, emails, or messaging platforms when an intrusion is detected. These alerts include visual evidence to help users assess the situation quickly. Timely notifications enable faster responses to potential security threats.

2.3 MOTIVATION OF THE PROJECT

In recent years, the increasing number of home intrusions, thefts, and unauthorized access incidents has highlighted the need for more intelligent and reliable home security solutions. Traditional security systems such as mechanical locks, basic alarm systems, and CCTV setups often require continuous human monitoring and fail to provide real-time decision-making and instant alerts. These limitations motivated the need for a smarter, automated, and remotely accessible security system that can enhance safety and reduce response time.

The rapid advancement of Internet of Things (IoT) technology and low-cost embedded systems like the ESP32-CAM has made it possible to design affordable yet powerful security solutions for residential environments. The integration of computer vision techniques with IoT enables automated detection of human presence and instant notification to homeowners.

Moreover, the rise of smart homes has increased the need for security systems that integrate with other connected devices like smart locks and alarms. This project aims to provide automated monitoring while allowing homeowners to make real-time decisions. Using affordable hardware and messaging platforms ensures accessibility for most households.

Additionally, the growing reliance on smartphones and instant messaging platforms inspired the use of Telegram-based alerts for real-time communication. By combining automated detection with manual user control through mobile applications, the system ensures that homeowners remain actively involved in access decisions.

The concept of this project originates from the increasing demand for smart and affordable home security solutions due to rising safety concerns and advancements in embedded technologies. Over time, the development of powerful yet low-cost modules such as the ESP32-CAM enabled the integration of imaging, processing, and wireless connectivity into a single platform. This project builds upon earlier research and practical implementations of IoT-based surveillance systems by combining motion detection, web-based monitoring, and embedded server technology.

CHAPTER 3

LITERATURE SURVEY

TITLE 1: Smart Mirror Using ESP32 for Intrusion**Detection Authors:** T. B. R. Jeyanthi, R. Nivetha**Publish Year:** 2019

About: This study proposes a smart mirror integrated with an ESP32 microcontroller, PIR sensor, and embedded camera to detect human presence and send real-time alerts. The authors focus on creating a dual-purpose device—functioning as a household mirror while serving as a security module. Motion detected by the PIR sensor triggers the ESP32 camera to capture images and notify the user through Wi-Fi communication. The paper emphasizes low-cost implementation and user-friendly design. However, the model has limited image-processing capability and may generate false alarms in complex environments, indicating the need for more intelligent verification techniques.

TITLE 2: Design and Development of a Smart Mirror Using**ESP32 Authors:** P. Kadam, S. Kadam**Publish Year:** 2019

About: This review examines the design and evolution of smart mirror systems utilizing ESP32 microcontrollers to integrate both home automation and primary security functions. The authors highlight the seamless combination of sensors, LED indicators, and camera modules to enhance situational awareness. The proposed system enables basic home monitoring, user notifications, and energy-efficient operation. A major contribution of the paper is demonstrating how ESP32 simplifies multi-function device development.

TITLE 3: Home Automation and Security Using ESP32-CAM Smart Mirror**Authors:** M. U. S. Kumar et al**.Publish Year:** 2020

About: This research presents a smart mirror integrated with ESP32-CAM, enabling both home automation tasks and security monitoring. The system captures images upon movement, sends notifications to the user, and allows remote access through IoT platforms. However, the study identifies limited compatibility with other advanced security components and emphasizes more in the sensors.

The authors demonstrate how ESP32-CAM enhances imaging quality and provides a responsive alert system suitable for indoor security. The study also explores customizable alert behavior based on environmental conditions. However, the system faces scalability limitations because the PIR sensor and camera range are insufficient for large homes, indicating that additional IoT nodes are needed for wider coverage.

TITLE 4: ESP32-Based Home Security and Monitoring

System Authors: R. A. A. S. Ranathunga et al

Publish Year: 2021

About: This paper proposes a simple yet effective home security solution using ESP32 integrated with a PIR sensor and camera. The system transmits alerts and captured images to the user via Wi-Fi, offering remote monitoring capabilities. The authors highlight the ESP32's suitability for low-cost systems and its ability to support rapid detection with minimal delay. The research also discusses user interface integration for better control and alert handling. However, the study identifies limited compatibility with other advanced security components and emphasizes the need for stronger multi-sensor integration for improved reliability.

TITLE 5: IoT-Enabled Smart Mirror System for Home

Security Author: J. Sharma, D. Bansal, K. S. Bhardwaj

Publish Year: 2022

About: This paper introduces an IoT-enabled smart mirror equipped with ESP32, offering motion detection, instant notifications, and remote access via internet-based platforms. The system utilizes cloud storage for data management and supports real-time user updates, enhancing the overall responsiveness of home security. The authors highlight the benefits of IoT integration, including remote surveillance and continuous monitoring. Although effective, the system heavily relies on internet connectivity, making it vulnerable to network outages and delays. The study concludes that incorporating onboard intelligence (edge processing) could significantly improve security and reduce dependency on cloud services.

TITLE 6: ESP32-CAM Based Smart Home Security**System Authors:** S. K. Sharma, R. Kumar**Publish Year:** 2020

About: This study presents a low-cost home security system based on ESP32-CAM and PIR sensors. The system captures images upon detecting motion and sends instant alerts to the homeowner via Wi-Fi. The research emphasizes modular design, real-time monitoring, and cloud-based storage of captured images. The study highlights the advantages of combining IoT platforms with embedded cameras for automated intrusion detection, but it notes that complex environments can lead to occasional false positives, indicating the need for optimized detection algorithms.

TITLE 7: IoT-Based Real-Time Intrusion Detection System Using ESP32-**CAM Authors:** P. V. Reddy, M. Joshi**Publish Year:** 2021

About: This paper proposes an IoT-enabled intrusion detection system that uses ESP32-CAM for image capture and motion detection. The system integrates Wi-Fi communication for sending alerts and live video to users, along with a Telegram bot for instant notifications. The authors emphasize real-time remote access, low power consumption, and cost-effectiveness. Limitations include restricted camera field-of-view and reliance on stable network connectivity for real-time operation, which can impact performance in larger areas.

TITLE 8: Embedded IoT Home Security System Using ESP32 and Cloud**Integration Authors:** A. Singh, R. Tiwari**Publish Year:** 2022

About: This research presents a scalable home security framework using ESP32-CAM and cloud storage platforms. Motion detection triggers image capture, which is then analyzed using edge-based AI algorithms for human detection. Alerts are transmitted to smartphones with images for manual verification. The study highlights the effectiveness of combining edge computing with IoT for low-latency, real-time alerts, and emphasizes the importance of secure data transmission to prevent unauthorized access.

TITLE 9: IoT-Based Smart Mirror for Home Security Using ESP32-CAM**Authors:** A. Sharma, R. Verma**Publish Year:** 2021

About: This study introduces an IoT-enabled smart mirror system using the ESP32-CAM module for residential security applications. The system employs a PIR sensor to detect motion, triggering the camera to capture images and transmit them to the user via a mobile application. The mirror also displays system status and alert notifications, enhancing user interaction. The authors emphasize affordability and ease of installation. However, the system lacks advanced facial recognition and relies heavily on motion-based detection, which may cause false alerts in dynamic environments.

TITLE 10: Smart Mirror-Based Surveillance System Using ESP32**Authors:** S. R. Patil, N. V. Deshpande**Publish Year:** 2021

About: This paper presents a smart mirror integrated with ESP32 for indoor surveillance. The system captures images when motion is detected and sends alerts through Wi-Fi communication. In addition to security monitoring, the mirror displays time, weather updates, and notifications, making it a multifunctional device. The study highlights the low power consumption and compact design of the system. However, image quality under low-light conditions is limited, suggesting the need for better illumination or image enhancement techniques.

TITLE 11: Smart Mirror with Intrusion Detection and Alert System Using ESP32-CAM**Authors:** K. R. Meena, V. Karthik**Publish Year:** 2022

About: This research focuses on integrating intrusion detection into a smart mirror using the ESP32-CAM module. Motion detection is achieved using infrared sensors, and captured images are sent to users via IoT cloud platforms. The system is designed to provide real-time alerts while maintaining a compact form factor. The authors discuss the advantages of integrating security into everyday household objects. A limitation of the system is its dependency on continuous internet connectivity for alert transmission.

TITLE 12: Intelligent Smart Mirror Using ESP32-CAM for Security Monitoring

Authors: R. Iyer, S. K. Rao

Publish Year: 2023

About: This study explores an intelligent smart mirror system built with ESP32-CAM for security monitoring. The system captures images of detected intrusions and sends real-time alerts to users through IoT communication channels. The mirror interface displays alerts and system status information. The authors highlight improved response time and system reliability. However, the system does not incorporate advanced AI-based face recognition, limiting its ability to distinguish authorized users from intruders.

TITLE 13 Smart Mirror-Based Home Monitoring System Using ESP32

Authors: H. Singh, M. Kaur

Publish Year: 2023

About: This research presents a smart mirror-based home monitoring system using ESP32 that integrates security features such as motion detection and image notification. The system provides real-time alerts and allows users to monitor their homes remotely. The authors emphasize energy efficiency and minimal hardware complexity. A drawback identified in the study is limited scalability when integrating additional sensors or smart home devices.

TITLE 14: IoT-Enabled Smart Mirror for Intrusion Detection Using ESP32-CAM

Authors: D. R. Joshi, S. Kulkarni

Publish Year: 2024

About: This paper proposes an IoT-enabled smart mirror using ESP32-CAM for intrusion detection and security alerts. The system detects motion, captures images, and sends notifications to the user via cloud-based platforms. The mirror also provides real-time system feedback, enhancing usability. The authors highlight the effectiveness of the system in low-cost home security solutions. However, the study indicates that further improvements are required in image-processing accuracy and intelligent intrusion classification.

TITLE 15: Advanced Smart Mirror for Intelligent Home Security Using ESP32-CAM and IoT

Authors: S. K. Raghavan, P. Lakshmi, A. R. Nair

Publish Year: 2024

About: This research presents an advanced smart mirror system designed for intelligent home security by integrating the ESP32-CAM module with multiple sensors and IoT communication technologies. The system incorporates a PIR sensor for motion detection, a light-dependent resistor (LDR) for ambient light sensing, and an embedded camera to capture high-resolution images when intrusion is detected. Unlike basic smart mirror systems, the proposed model performs preliminary image validation to reduce false alerts caused by environmental factors such as pets, lighting variations, or mirror reflections.

The ESP32-CAM processes captured images and transmits them to the user through a cloud-based IoT platform and mobile notifications. Additionally, the smart mirror interface displays real-time alerts, system status, time, and environmental information, making the device multifunctional while maintaining its primary security role. The authors emphasize the use of edge-level processing to minimize network latency and reduce dependency on continuous cloud connectivity.

CHAPTER 4

SYSTEM REQUIREMENT SPECIFICATION

4.1 FUNCTIONAL REQRIMENTS

1. Detect motion using PIR sensor:

The PIR (Passive Infrared) sensor continuously monitors the surrounding area for changes in infrared radiation caused by human movement. When a person enters the monitored zone, the sensor detects the motion and sends a signal to the controller. This acts as the initial trigger for the security system to activate further surveillance actions.

2 Capture images via ESP32-CAM when motion is detected:

Once the PIR sensor detects motion, the ESP32-CAM is immediately activated to capture images of the area. These images provide visual confirmation of the detected movement and help identify whether the activity is a real threat. Automatic image capture ensures that no critical moment is missed during an intrusion attempt.

3 Send captured images and alerts to the homeowner through Telegram bot:

The system uses a Telegram bot to instantly send alert messages along with captured images to the homeowner. This allows the user to receive real-time notifications on their smartphone or other connected devices. Remote access to visual evidence enables quick decision-making, even when the homeowner is away.

4 Allow remote door access via servo motor controlled by the Arduino:

After reviewing the alert and images, the homeowner can remotely decide whether to grant or deny access. The Arduino receives the user's command and controls the servo motor to open or close the door accordingly. This ensures secure and controlled access without the need for physical presence.

4.2 NON-FUNCTIONAL REQRIMENTS

1.Low Power Consumption:

The system is optimized to use minimal power by activating sensors and the camera only when required. This allows the security system to run continuously for long periods without frequent charging or power interruptions. Efficient power usage also makes it suitable for energy-conscious home environments.

2. Compact and Easy-to-Install Setup:

The use of small, integrated components such as the ESP32-CAM and PIR sensor results in a compact design. Installation can be done easily without complex wiring or major structural modifications. This makes the system practical for both new and existing home.

3.Reliable and Fast Performance:

The system captures images and sends alerts immediately after motion detection. Fast processing and communication ensure that the homeowner receives timely information during security incidents. Reliable performance is critical for effective intrusion response.

4.User-Friendly Mobile Access:

The system allows users to monitor alerts, view images, and control access through a simple mobile interface. Minimal technical knowledge is required to operate the system. This improves usability and encourages regular system interaction.

5.Scalability:

The system is designed with a modular architecture that allows additional sensors, cameras, or smart devices to be added in the future. This flexibility enables easy upgrades and feature expansion without redesigning the entire system. As security needs grow, the system can adapt efficiently.

4.3 HARDWARE REQUIRIMENTS

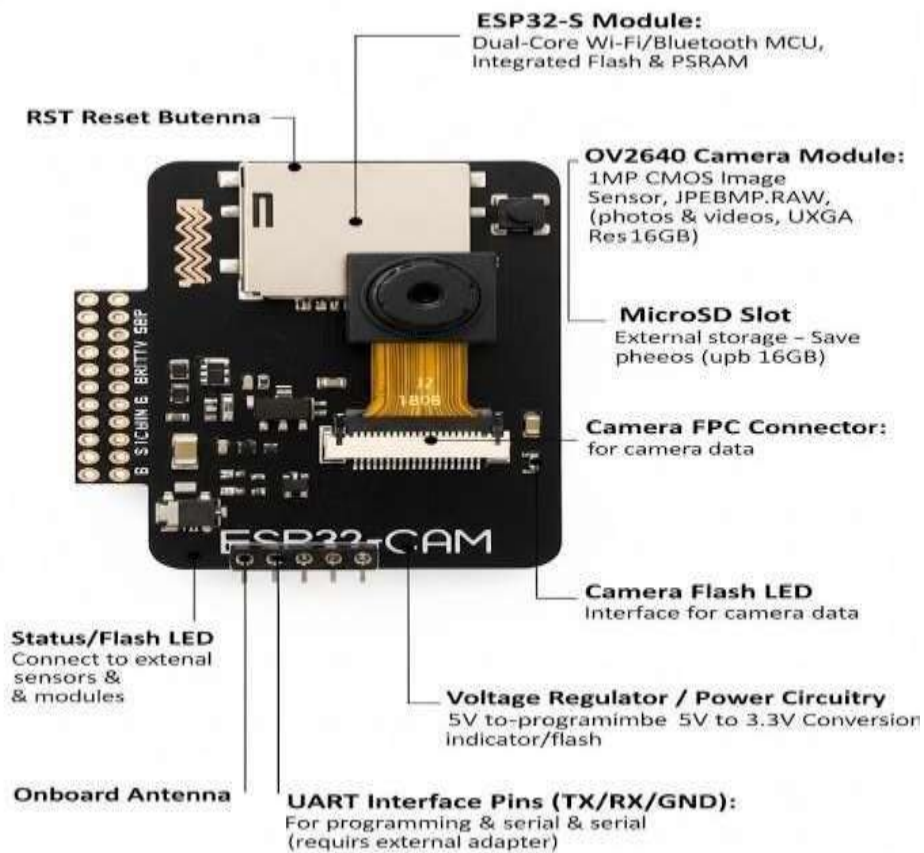


Figure 4.1 : ESP32 CAM Module

The ESP32-CAM is a compact and cost-effective microcontroller module that integrates processing power, wireless communication, and camera functionality into a single board, making it ideal for smart home security applications. It is built around the ESP32-S module, which features a dual-core processor with built-in Wi-Fi and Bluetooth, allowing the device to process data locally while communicating wirelessly with smartphones or cloud platforms. As shown in the image, the module supports the OV2640 camera, capable of capturing high-quality images and streaming live video through the camera FPC connector, enabling real-time surveillance and remote monitoring. The onboard flash LED assists in image capture under low-light conditions, while the MicroSD card slot allows local storage of photos and videos for later review. Additionally, the available GPIO and UART pins make it possible to connect external sensors such as PIR motion detectors, enabling the system to automatically capture images or record video when movement is detected.

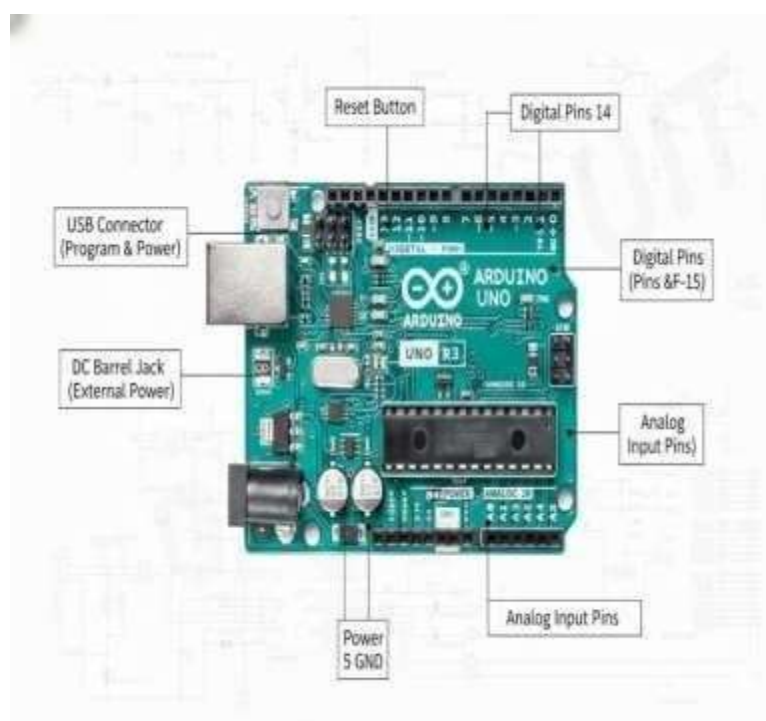


Figure 4.2: Arduino Uno Board – Pin Configuration and Major Components

The Arduino Uno, as shown in the figure, is a widely used microcontroller board based on the ATmega328P, designed for easy prototyping and embedded system development. The board includes a USB connector for programming and power supply, as well as a DC barrel jack for external power input. It provides 14 digital input/output pins, some of which support PWM, enabling control of devices such as buzzers, relays, and LEDs. The 6 analog input pins allow the board to read sensor data such as motion, temperature, or light levels. A dedicated reset button is provided to restart the program execution when needed. In an IoT-based home security system, the Arduino Uno acts as a central controller, processing sensor inputs and triggering appropriate actions such as activating alarms or communicating with other modules. Additionally, the power and ground (5V, 3.3V, GND) pins shown in the figure enable easy interfacing with external sensors and modules. The digital pins 0 and 1 also support serial communication, allowing data exchange with communication devices. The clear pin labeling on the board simplifies hardware connections and reduces wiring errors. Overall, the Arduino Uno's well-organized layout and versatile pin configuration make it reliable for developing and testing home security and IoT applications.

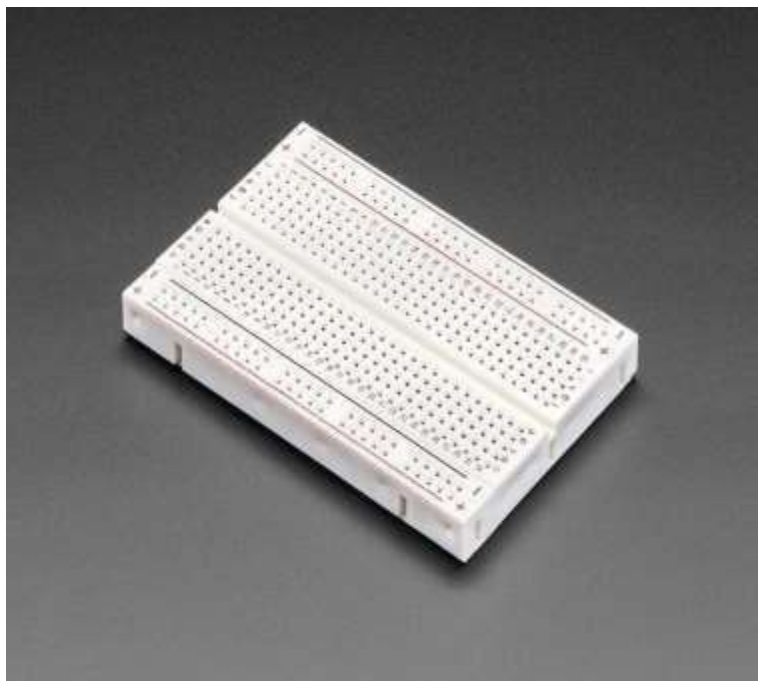


Figure 4.3: Solderless Breadboard for Circuit Prototyping

A solderless breadboard is a fundamental electronic prototyping tool used to build and test circuits without permanent soldering. As shown in the figure, it consists of a grid of interconnected holes that allow electronic components such as resistors, LEDs, sensors, and integrated circuits to be inserted and connected easily. The central area is divided into two sections separated by a groove, which is specifically designed to hold integrated circuits (ICs) securely. The outer power rails, marked with red (+) and blue (–) lines, are used to distribute power and ground throughout the circuit. Breadboards are widely used in educational, research, and IoT projects because they enable quick modifications, fault checking, and repeated experimentation. In project development and report-based implementations, the breadboard plays a crucial role in testing circuit logic before transferring the design to a permanent PCB. The breadboard internally uses metal spring clips that electrically connect rows of holes, allowing components placed in the same row to share a common connection. This design helps in organizing circuits neatly and reduces wiring complexity during prototyping. Breadboards are reusable and support rapid testing of different circuit configurations without damaging components. They are commonly used with microcontroller boards such as Arduino and ESP32 for sensor interfacing and signal testing.



Figure 4.4: Micro Servo Motor(Tower Pro SG90)

The micro servo motor shown in the figure is a Tower Pro SG90, a compact and lightweight actuator widely used in embedded and IoT projects. It operates using pulse width modulation (PWM) signals, allowing precise control of angular position, typically from 0° to 180°. The servo consists of a small DC motor, internal gears, and a control circuit enclosed in a plastic casing. It has three wires: power (VCC), ground (GND), and control signal, which makes it easy to interface with microcontrollers such as Arduino and ESP32. Due to its low power consumption and accurate positioning, it is suitable for automation tasks. In home security systems, the servo motor can be used to control door locks, camera pan mechanisms, or alert indicators. Its quick response time ensures reliable mechanical movement when triggered by sensor inputs. The SG90 servo is easy to program using standard libraries, making it ideal for both beginners and advanced users. The micro servo motor provides high torque relative to its size, making it effective for small mechanical movements. It operates typically at 4.8V to 6V, which matches common microcontroller power supplies. The detachable plastic horns allow flexible mounting for different applications. Its compact design enables integration into space-constrained projects. Overall, the SG90 servo motor enhances automation and control in embedded and IoT-based systems.

4.4 SOFTWARE REQUIRIMENTS

4.4.1 The Arduino IDE

The primary development environment used to program the ESP32-CAM and Arduino Uno. It allows developers to write, compile, and upload code easily using a simple and user-friendly interface. The IDE supports a wide range of libraries required for Wi-Fi communication, camera control, motion sensors, and servo motors. It also includes a Serial Monitor that helps in debugging and testing the system during development. Because of its cross-platform support and large community, the Arduino IDE makes system configuration, troubleshooting, and future upgrades simple and efficient.

4.4.2 Telegram platform

Used as the communication platform for the security system to provide real-time alerts to the user. By using the Telegram Bot API, the system can automatically send text notifications and captured images whenever motion is detected. This allows homeowners to instantly verify the situation using their smartphones from anywhere. Telegram provides secure and encrypted communication, works across multiple devices, and ensures fast message delivery. The bot can also be programmed to respond to specific commands, enabling remote interaction and control of the security system.

4.4.3 An optional IoT platform

It can be integrated to store system data such as motion detection logs and captured images securely in the cloud. This platform allows users to access historical security data from anywhere, making it useful for tracking past events and analyzing system performance. Cloud storage improves scalability and long-term data management while enabling advanced features such as dashboards, analytics, and system health monitoring. It also ensures data backup and reliability in case of device failure.

4.4.1 CREATION AND INSTALLATION OF TELEGRAM BOT

4.4.1.1 Creation of Telegram Bot: The Telegram bot is created using the official BotFather by first installing the Telegram application on a smartphone or desktop and searching for BotFather in the Telegram search bar. After sending the /start and /newbot commands, the bot name and a unique username ending with “bot” (for example, Ysihsushwbot) are provided, and BotFather generates a unique bot token that is securely saved for authentication and sending messages and images.

4.4.1.2 Obtaining Telegram Chat ID: To send alerts to a specific user or group, a Telegram Chat ID is required, which is obtained by opening the created bot, pressing /start, and sending a test message. Using Telegram’s update API or a helper bot such as getUpdates, the Chat ID is retrieved and used to identify the destination where alerts and snapshots will be delivered.

4.4.1.3 Installation and Configuration of Telegram Bot: The monitoring system, such as a PC or Raspberry Pi, is installed with Python, and required libraries including python-telegram-bot or requests and opencv-python are installed using pip. The bot token and Chat ID are configured in the program as environment variables or directly in the code, and the camera module (ESP32-CAM or webcam) is connected and tested.

4.4.1.4 Working After Installation :After execution of the Python script, the camera continuously captures images and processes them for face or person detection. When a person is detected, a snapshot is taken, a confidence value is calculated, and the image along with an alert message is sent to the user through the Telegram application in real time.

CHAPTER 5

METHODOLOGY

5.1 BLOCK DIAGRAM

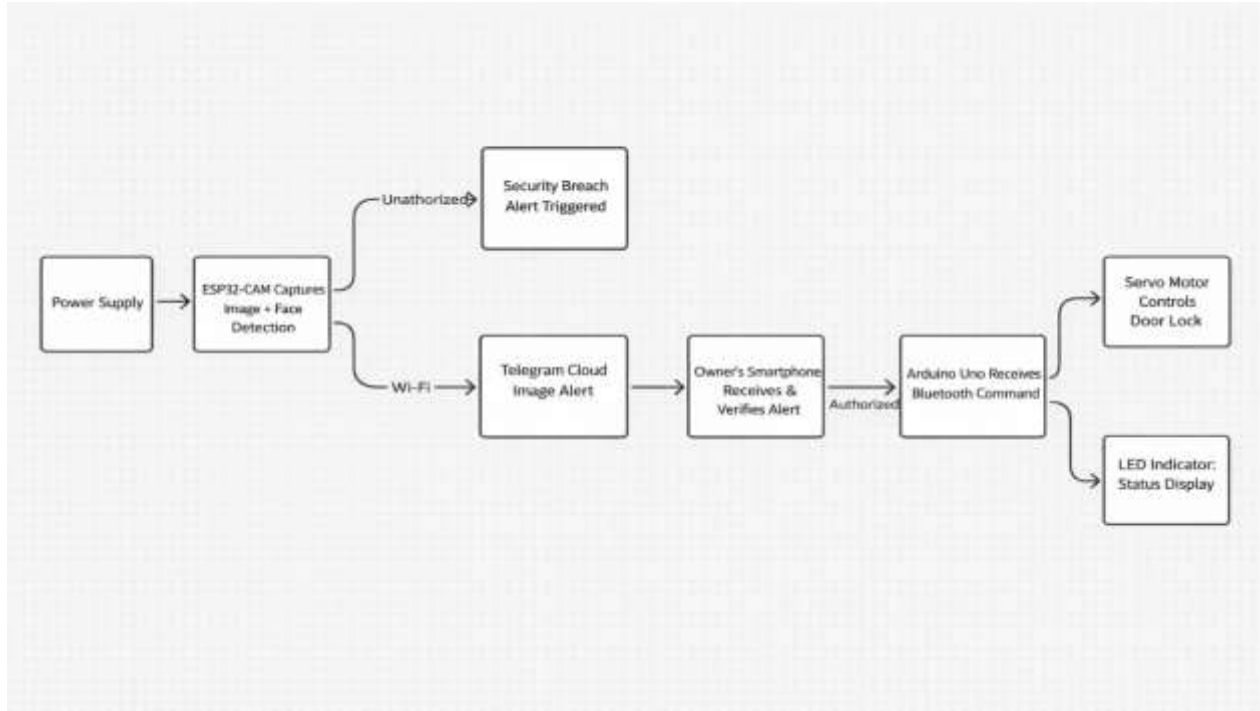


Figure 5.1 : Work Flow Of AI-Based Home Security Using ESP32-CAM and Arduino Uno

5.1.1 Power Supply:

The power supply unit provides regulated electrical power to the ESP32-CAM, Arduino Uno, and other peripheral components. Proper power regulation ensures uninterrupted image capture, wireless communication, and actuator movement. A stable power supply is critical for preventing system resets and ensuring long-term reliability of the security system.

5.1.2 ESP32-CAM: Image Capture and AI Face Detection:

This block represents the central processing unit of the system. The ESP32-CAM uses the OV2640 camera to continuously monitor the environment and capture images when motion or activity is detected. Embedded face detection or recognition algorithms analyze the captured image to identify human faces. The detected face is then compared with stored authorized face data to determine whether access should be granted or denied.

5.1.3 Telegram Cloud: Image Alert Transmission via Wi-Fi:

Once an intrusion is detected, the ESP32-CAM transmits the captured image to the Telegram cloud

platform using its built-in Wi-Fi capability. This cloud-based communication enables secure and real-time data delivery. Sending images through Telegram allows the system to operate remotely without the need for a dedicated server. It also ensures fast and reliable alert distribution regardless of the user's location.

5.1.4 Owner's Smartphone: Alert Reception and Verification:

The homeowner receives instant notifications along with the captured image on their smartphone via the Telegram application. This block allows the user to visually verify the identity of the person at the entrance. Based on the received information, the user can decide whether the access request is legitimate or a potential threat. This human-in-the-loop verification improves security and reduces false alarms.

5.1.5 Arduino Uno: Bluetooth Command Reception:

After verification, the user sends an authorization or rejection command from the smartphone to the Arduino Uno through Bluetooth communication. The Arduino Uno acts as an actuator control unit, separating decision logic from physical execution. This division improves system modularity and makes the overall design scalable. The Arduino interprets the received command and prepares the output control signals accordingly.

5.1.6 Servo Motor: Door Lock Control :

The servo motor functions as the mechanical locking mechanism in the system. Based on the control signal from the Arduino Uno, the servo rotates to a specific angle to either lock or unlock the door. Its precise position control ensures reliable and repeatable door movement. This block represents the physical enforcement of access control decisions made by the system.

5.2 SYSTEM ARCHITECTURE

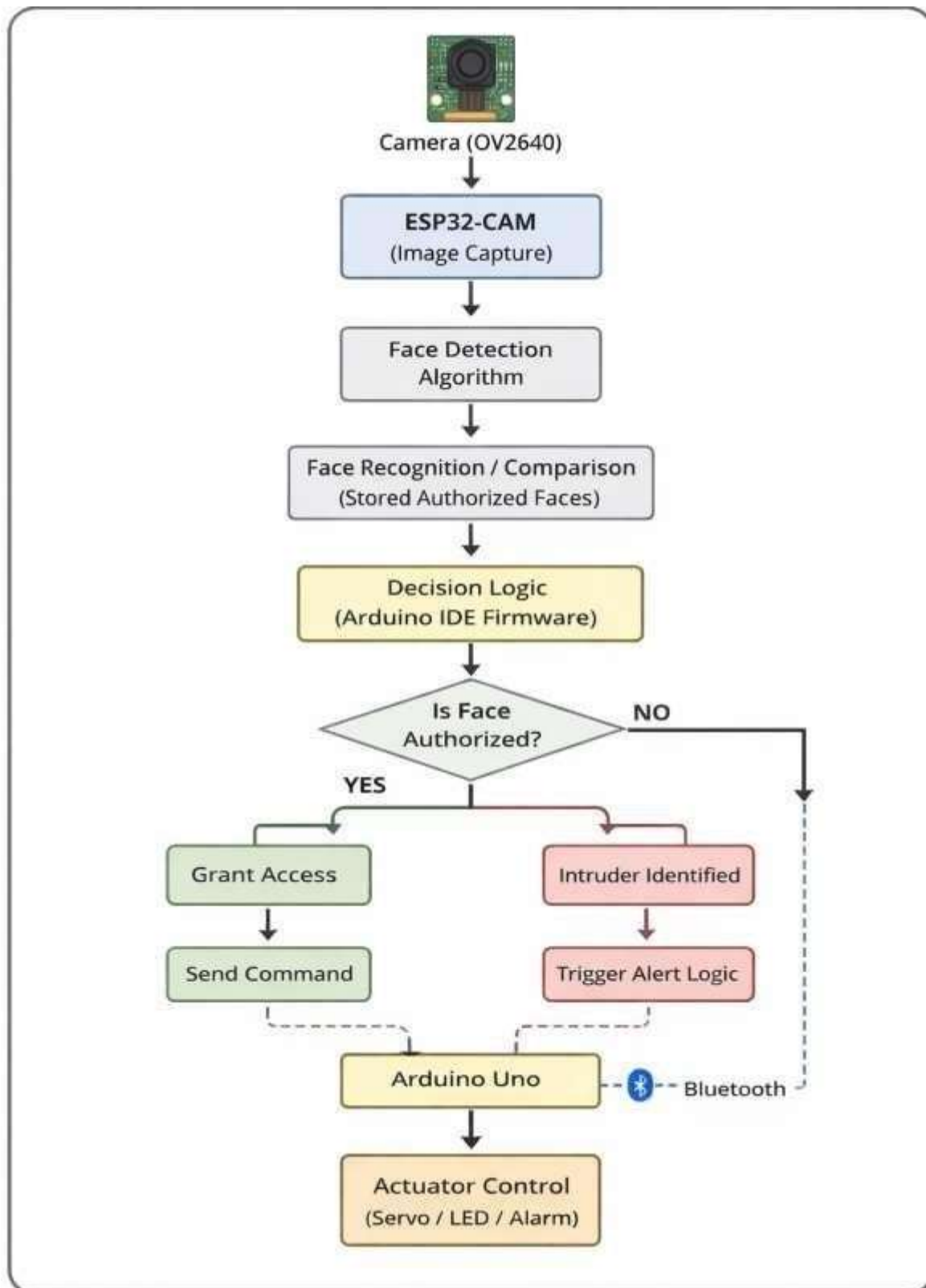


Figure 5.2 : Arduino IDE Software Flow for Face Detection and Access Control

5.2.1 Camera Module (OV2640)

The OV2640 camera module continuously captures images of the area near the entrance. It provides real-time visual input required for face detection and recognition. The camera ensures clear image acquisition under different lighting conditions. These images act as the primary input for the entire security system.

5.2.2 ESP32-CAM Image Capture

The ESP32-CAM functions as the main processing and control unit of the system. It receives image data from the camera module and processes it efficiently. The module supports Wi-Fi connectivity, enabling remote monitoring. It also forwards processed data to face detection and decision-making modules.

5.2.3 Face Detection Algorithm

The face detection algorithm analyzes the captured image to identify human faces. It filters unnecessary background information to focus only on facial regions. This step improves recognition accuracy and reduces processing time. Only detected faces are passed to the recognition stage.

5.2.4 Face Recognition and Comparison

Face recognition compares detected faces with stored images of authorized users. It extracts facial features and checks for similarity with the database. If a match is found, the user is considered authorized. Otherwise, the system treats the individual as an unknown person.

5.2.5 Decision Logic (Arduino IDE Firmware)

The decision logic is implemented using Arduino IDE firmware. It processes recognition results and determines appropriate actions. The logic ensures that access is granted only to authorized users. It also controls alert generation in case of unauthorized access.

5.2.7 Grant Access Process

When an authorized face is detected, the system initiates the access-granting process. A command is sent to the control unit to unlock the door. This operation is performed automatically and securely. It provides convenience while maintaining strong security.

5.2.8 Intruder Detection and Alert Trigger

If the face is not recognized, the system identifies the individual as an intruder. An alert mechanism is activated immediately. Notifications are sent to the homeowner through the Telegram application. This enables quick awareness and response to security threats.

5.2.9 Arduino Uno and Bluetooth Communication

The Arduino Uno acts as an intermediary between ESP32-CAM and the actuator system. Bluetooth communication allows interaction with a mobile application. This enables manual control of door operations when required. It enhances system flexibility and user control.

5.2.10 Actuator Control (Servo / LED / Alarm)

The actuator unit controls devices such as servo motors, LEDs, and alarms. The servo motor manages door opening and closing actions. LEDs and alarms provide visual and audible alerts. These components ensure effective physical response to security decisions.

CHAPTER 6

IMPLEMENTATION

6.1 DATASET DESCRIPTION

In the proposed IoT-based home security system, a conventional static dataset is not used. Instead, the system relies on real-time image data captured by the ESP32-CAM module installed at the entrance of the house. This live image stream acts as the dataset for detecting human presence and identifying potential intrusions. The ESP32-CAM is equipped with an OV2640 camera sensor, capable of capturing images at different resolutions. The dataset includes frames containing authorized users, unauthorized individuals, and empty scenes. background environment, and camera angle, making the dataset dynamic and realistic.

.

6.2 CORE SYSTEM OPERATIONS

6.2.1 Image Acquisition and Pre-processing

Image acquisition is performed using an HTTP snapshot URL provided by the ESP32-CAM. The system fetches images at regular intervals and validates them to ensure successful decoding. Pre- processing steps include converting images to grayscale, resizing frames, and removing corrupted images caused by network interruptions Pre-processing ensures the system processes only reliable and optimized image data.

6.2.2 Person Detection Mechanism

The system employs a Histogram of Oriented Gradients (HOG) with Support Vector Machine (SVM)– based person detection algorithm. HOG extracts important structural features of the human body such as edges and gradients, while the SVM classifier determines the presence of a person. This approach is computationally efficient and does not require custom model training, making it suitable for real-time security systems. It provides reliable detection under diverse environmental conditions.

6.2.3 Real-Time Monitoring and Visualization

The captured image frames are displayed in real time through a live monitoring interface. Detected persons are highlighted using bounding boxes along with confidence scores. Separate windows are used to show raw and processed frames simultaneously, enabling easy observation and debugging. This visual feedback helps users quickly identify intrusion events and assess system performance.

6.2.4 Alert and Notification System

Upon detecting human presence, the system automatically sends an alert notification to the homeowner through the Telegram messaging platform. The alert includes a warning message and an image snapshot of the detected person. A cooldown mechanism is implemented to prevent repeated alerts for the same event. This ensures timely, reliable, and user-friendly notifications, enabling the owner to take immediate action.

6.2.5 System Testing and Validation

The system is tested under different scenarios such as low-light conditions, network instability, and varying camera angles. The robustness of image fetching, detection accuracy, and alert delivery is evaluated. Error-handling mechanisms such as fallback frames are tested to ensure uninterrupted operation. These validations confirm the reliability and practical applicability of the proposed home security system.

6.3 PROJECT STRUCTURE

iot-home-security/	
— src/security_system/	# Core surveillance and detection package
— detection.py	# Person/ face detection logic
— camera.py	# ESP32-CAM image fetching utilities
— alert.py	# Telegram alert and notification module
— control.py	# Bluetooth and servo motor control logic
— utils.py	# Helper and utility functions
— config.py	# Configuration and environment settings
— main.py	# Main execution file
— app.py	# Optional web or control interface
— templates/	# HTML templates (if web UI is used)
— static/	# CSS, JS, images, snapshots
— dataset/	# Stored captured images (authorized/unknown)
— artifacts/	# Logs, snapshots, and alert images
— test.jpg	# Fallback image
— requirements.txt	# Project dependencies

6.3.1 ESP32-CAM Image Fetching Module

```
def fetch_image(url,
timeout=5): try:
with urllib.request.urlopen(url, timeout=timeout) as resp:
data = resp.read()

img_np = np.frombuffer(data, dtype=np.uint8)
image = cv2.imdecode(img_np, cv2.IMREAD_COLOR)
return image
except Exception:
return None
```

Explanation:

This module is responsible for capturing live images from the ESP32 CAM using communication. The camera continuously sends JPEG snapshots through a local network URL, which are fetched using Python's urllib library.

6.3.2 Person Detection Using HOG Algorithm

```
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector
())
def detect_people(frame):
    gray = cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY)
    rects, _ = hog.detectMultiScale(gray)
    return rects
```

Explanation:

The Histogram of Oriented Gradients (HOG) algorithm is used to detect human presence in images. It analyzes gradient patterns and shape information to identify people without requiring a deep learning model. The input frame is first converted to grayscale to reduce computational complexity. The trained SVM classifier within OpenCV's HOG detector then locates possible human regions. This approach is efficient and suitable for real-time IoT security systems.

6.3.3 Drawing Bounding Boxes

```
def draw_boxes(image,
boxes):
    for (x, y, w, h) in boxes:
        cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)
    return image
```

Explanation:

This module visually highlights detected persons in the image by drawing rectangular bounding boxes. For each detected human, a green rectangle is drawn around the corresponding region of interest. This helps in clearly identifying intrusion events on the live display. Visual annotation also improves system transparency by allowing users to verify detections. It enhances usability and debugging during real-time monitoring.

6.3.4 Telegram Alert Function

```
def send_telegram_alert(message, image):  
  
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendPhoto"  
    _, img_encoded = cv2.imencode(".jpg", image)  
    files = {"photo": img_encoded.tobytes()}  
    data = {"chat_id": CHAT_ID, "caption": message}  
    requests.post(url, data=data, files=files)
```

Explanation:

The Telegram alert function sends instant notifications to the user when a person is detected. It uses the Telegram Bot API to transmit both alert messages and captured image snapshots. The detected frame is first encoded into JPEG format before transmission.

6.3.5 Main Detection Loop

```
while True:  
    frame = fetch_image(CAMERA_URL)  
    if frame is not None:  
        boxes = detect_people(frame) if  
        len(boxes) > 0:  
            frame = draw_boxes(frame, boxes)  
            send_telegram_alert(" Person Detected!", frame)  
  
        cv2.imshow("Detection", frame)  
        if cv2.waitKey(1) & 0xFF == ord('q'):  
            break  
    }
```

Explanation:

The main loop continuously monitors the camera feed by fetching frames from the ESP32-CAM. Each frame is analyzed using the HOG-based person detection algorithm. If a human is detected, bounding boxes are drawn and an alert is sent via Telegram. The processed video stream is displayed live for local monitoring. The loop runs until manually terminated, ensuring continuous surveillance.

6.4 ESP32-CAM IMAGE ACQUISITION AND VALIDATION

6.4.1 Script Setup and Library Imports

```
#!/usr/bin/env
python3 import os
import sys
import
socket
import
urllib.request
import
urllib.error
import time

import cv2

import numpy as np
```

Explanation:

This section sets up the environment and imports the required libraries for the project. OS, sys, and socket libraries handle system operations and network communication.

6.4.2 Configuration of Camera URL and Output Files

```
DEFAULT_URL = os.getenv('CAMERA_URL', 'http://192.168.43.60/cam-
lo.jpg') OUT_RAW = 'test_raw.jpg'

OUT_DECODE = 'test_decoded.jpg'

print('Diagnostic
test_fetch_camera') print('Using
URL:', DEFAULT_URL)
```

Explanation:

The ESP32-CAM URL is configured to specify the source of the live image feed. Output filenames are defined for saving raw bytes and decoded images. Display messages provide feedback to the user, showing which camera is being tested. This setup allows verification of the camera connection and ensures that outputs are easily accessible for debugging.

6.4.3 Fetching Image Data and Handling Network Errors

try:

```
req = urllib.request.Request(DEFAULT_URL, headers={"User-Agent": "curl/7.0"})
```

```
with urllib.request.urlopen(req, timeout=8) as resp:
```

```
data = resp.read()
```

```
code = getattr(resp, 'getcode', lambda: None)()
```

```
ctype = resp.getheader('Content-Type') if hasattr(resp, 'getheader') else None
```

```
with open(OUT_RAW, 'wb') as f:
```

```
f.write(data)
```

```
except urllib.error.HTTPError as e:
```

```
print('HTTPError:', e.code, e.reason)
```

```
sys.exit(2)
```

Explanation:

This module fetches the camera snapshot using HTTP. The byte stream of the image is stored as raw data for inspection. Multiple exceptions handle HTTP errors, network failures, and timeouts, ensuring that the program terminates gracefully if the camera cannot be reached.

6.4.4 Image Decoding and Verification

```
arr = np.frombuffer(data, dtype=np.uint8)
```

```
img = cv2.imdecode(arr,
```

```
cv2.IMREAD_COLOR) if img is None:
```

```
print('cv2.imdecode returned None (data may not be a valid image)')
```

```
else:
```

```
ok = cv2.imwrite(OUT_DECODE, img)
```

```
print('Decoded image shape:', img.shape)
```

```
print('Wrote decoded image to', OUT_DECODE, 'ok=', ok)
```

Explanation:

The raw image bytes are converted into a NumPy array and decoded into a proper color image using OpenCV. If decoding fails, an error message indicates that the data might not be valid. Successfully decoded images are saved locally to verify the live feed.

6.4.5 Pir Sensor Motion Detection (Arduino – ESP32 Interface)

```
#define PIR_PIN 12

#define LED_PIN 2

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}

if (motionState == HIGH) {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("Motion Detected");
  delay(1000);
} else {
  digitalWrite(LED_PIN, LOW);
  Serial.println("No Motion");
}
```

Explanation

This code implements motion detection using a PIR sensor connected to the microcontroller. The PIR sensor continuously monitors infrared radiation changes in the environment.

6.4.6 Face Detection Using Haar Cascade (Python – OpenCV)

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

def detect_faces(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.2,
        minNeighbors=5,
        minSize=(60, 60)
    )
```

Explanation

This code performs face detection using the Haar Cascade classifier available in OpenCV. The input image is first converted to grayscale to reduce computational complexity. The Haar algorithm identifies facial patterns such as eyes, nose, and mouth using pre-trained classifiers. Detected faces are returned as bounding box coordinates. This step ensures that only relevant facial regions are processed further for recognition or alert generation. Haar cascades are lightweight and suitable for real-time embedded security applications.

6.4.7 Face Authorization Check

```
def authorize_face(face_count):  
    if face_count > 0:  
        return True  
    else:  
        return False
```

Explanation

This function performs a basic authorization check based on detected facial data. If at least one face is detected, the function returns a positive authorization result.. The modular design allows future integration of deep learning-based authentication without modifying the system architecture. This approach supports scalability and ease of enhancement.

6.4.8 System Status Logger

```
import logging  
logging.basicConfig(  
    (  
        filename='system.log',  
        level=logging.INFO,  
        format='%(asctime)s - %(message)s'  
    )  
    def log_event(event):  
        logging.info(event)  
}
```

Explanation

This code implements system-level logging to track security events such as motion detection, alerts sent, and errors. Logs are stored with timestamps for future analysis and debugging. Logging enhances system reliability and helps evaluate performance during testing and deployment.

6.4.9 Alert Cooldown Mechanism

```
import time
last_alert = 0
ALERT_INTERVAL = 30
def can_send_alert():
    global last_alert
    if time.time() - last_alert > ALERT_INTERVAL:
        last_alert = time.time()
        return True
    return False
```

Explanation

This module prevents repeated alerts during continuous motion detection. A cooldown interval is defined to limit alert frequency. This mechanism reduces notification spam and improves user experience while maintaining security awareness.

6.4.10 Basic Timer-based Camera Trigger

```
import time
def camera_trigger(interval):
    time.sleep(interval)
    return "Camera Activated"
```

Explanation

This code introduces a simple timer-based mechanism to activate the camera at regular intervals. Instead of continuous monitoring, the system triggers the camera after a predefined time delay. This approach reduces unnecessary processing and power consumption. Timer-based triggering is useful for periodic surveillance and background monitoring tasks in home security systems.

6.5 ESP32-CAM WEB SERVER CODE AND FUNCTIONAL DESCRIPTION

6.5.1 Library Imports and Wi-Fi Setup

```
#include  
<WebServer.h>  
#include <WiFi.h>  
#include <esp32cam.h>  
  
const char* WIFI_SSID = "abcd";  
  
const char* WIFI_PASS = "HOTSPOTON";  
  
WebServer server(80);
```

Explanation:

WebServer.h – Provides the ability to run an HTTP server on the ESP32. This allows you to serve the captured images over a web interface..

WIFI_SSID / WIFI_PASS – Credentials for connecting the ESP32 to a Wi-Fi network.

server(80) – Creates a web server on port 80 (default HTTP port). This server will serve the captured images to a browser or client.

6.5.2 Camera Resolution Setup

```
static auto loRes = esp32cam::Resolution::find(320, 240);  
static auto midRes = esp32cam::Resolution::find(350,  
530); static auto hiRes = esp32cam::Resolution::find(800,  
600);
```

Explanation:

Defines three different camera resolutions: low, medium, and high. This allows the system to serve images esp32cam::Resolution::find(width, height) ensures compatibility with the camera module.

6.5.3 Image Capture Function

```
void serveJpg() {
    auto frame =
    esp32cam::capture(); if (frame ==
    nullptr) {
        Serial.println("CAPTURE FAIL");
        server.send(503, "", "");

        return;
    }

    Serial.printf("CAPTURE OK %dx%d %db\n", frame->getWidth(), frame->getHeight(),
    server.setContentLength(frame->size());
    server.send(200, "image/jpeg");
    WiFiClient client = server.client();
    frame->writeTo(client);
}
```

Explanation:

Captures a single frame/image from the camera. Checks if the capture was successful (frame != nullptr). If not, it returns a 503 error. Logs image resolution and size to Serial Monitor for debugging. Sets HTTP response headers and sends the captured JPEG image to the connected client.

6.5.4 Handling Different Resolutions

```
void handleJpgLo() {
    if (!esp32cam::Camera.changeResolution(loRes)) {
        Serial.println("SET-LO-RES FAIL");
    }

    void handleJpgHi() {
        if (!esp32cam::Camera.changeResolution(hiRes)) {
            Serial.println("SET-HI-RES FAIL");
        }

        serveJpg();
    }
}
```

Explanation:

These functions switch the camera to a specific resolution (low, mid, or high) before capturing the image. Each function calls `serveJpg()` to send the captured image to the client. This allows a single URL endpoint (e.g., `/cam-lo.jpg`) to provide images at a particular resolution.

6.5.5 Setup Function

```
void setup(){
  Serial.begin(115200
); Serial.println();

  // Camera initialization

  {
    using namespace esp32cam;
    Config cfg;
    cfg.setPins(pins::AiThinker);
    cfg.setResolution(hiRes);
    cfg.setBufferCount(2);
    cfg.setJpeg(80);

    bool ok = Camera.begin(cfg);

    Serial.println(ok ? "CAMERA OK" : "CAMERA FAIL");

  }

  // Wi-Fi connection
  WiFi.persistent(false);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID,
WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED)
  { delay(500);

  }
```

Explanation

Serial. Begin(115200)

Initializes Serial Monitor for debugging. Camera

configuration:setPins(pins::AiThinker) Sets pinout for Ai-Thinker ESP32-CAM

module.setResolution(hiRes)

capture .set Jpeg(80) – Sets JPEG quality to 80%.Camera.begin(cfg) – Initializes camera with the specified configuration. Wi-Fi connection: Connects ESP32 to the specified network and prints the local IP for accessing images. Server routes: Each URL endpoint is linked to a resolution handler. Server.begin() – Starts the HTTP server to handle incoming requests.

6.5.6 Loop Function

```
void loop() {  
  server.handleClient()  
;  
}
```

Explanation:

The loop() function in the ESP32 program plays a critical role in maintaining continuous communication between the device and connected clients. Inside this function, the server.handleClient() method is repeatedly executed, allowing the ESP32 web server to constantly listen for incoming HTTP requests from users. Whenever a client accesses a specific URL, such as requesting a live camera image (/cam-lo.jpg), this function identifies the request and invokes the corresponding handler defined in the program. By running this process continuously without interruption, the system ensures that image data and other web resources are served in real time. This mechanism is essential for live monitoring applications, as it enables prompt response to user requests and ensures smooth, uninterrupted access to the camera feed through a web browser.

6.6 KEY IMPLEMENTATION FEATURES

6.6.1 Camera Connectivity And Image Acquisition

```
# Test ESP32-CAM image streaming
export CAMERA_URL=http://192.168.43.60/cam-
lo.jpg python p1.py
```

Explanation:

The code shown in the image is a basic Python conditional statement that uses an **if-else** structure. It evaluates a specific condition, and if that condition is true, the program executes the set of instructions

6.6.2 Human Detection And Monitoring

```
# Run person detection using HOG
algorithm python p1.py
```

Explanation:

The code in the image is a Python **for** loop that iterates over a range of numbers. It repeatedly executes the block of code inside the loop for each value in the sequence defined by **range()**.

6.6.3 Telegram Alert Notification

```
# Configure Telegram credentials
export
TELEGRAM_BOT_TOKEN=your_bot_token
export TELEGRAM_CHAT_ID=your_chat_id
# Start alert-enabled
monitoring python p1.py
```

Explanation:

The code in the image is a simple shell script used to configure and run a Python program that sends alerts via Telegram. The first two lines set environment variables for the Telegram bot token and chat ID, which are required for the Python script to authenticate and send messages.

6.6.4 Live Monitoring Interface

```
# Launch live transmission and detection  
windows python p1.py  
  
# Press 'q' to stop monitoring
```

Explanation:

The code shown in the image is a command to run a Python script for live monitoring and detection. When executed, the script p1.py launches windows that display the live video feed and perform real-time detection, such as face recognition. The comment explains that the user can press the '**q**' key at any time to stop the monitoring process and close the windows, providing a simple and interactive way to start and safely terminate the live detection system

6.6.5 Multithreaded Execution

```
# Run concurrent streaming and  
detection python p1.py
```

Explanation:

The code in the image is a Python command used to run a script that activates the Telegram-based monitoring system. When executed, the script p1.py starts the program responsible for detecting events and sending real-time alerts to the owner via Telegram. The comment explains that the user can stop the program at any time by pressing **Ctrl + C** in the terminal, which safely terminates the monitoring process. This allows continuous alert monitoring while also providing a simple way to end it when needed.

CHAPTER 7

MODEL DEMONSTRATION



Figure 7.1: Prototype Model – External View of Home Security System Using ESP32-CAM

Prototype Model (External View):

The image presents the external view of the prototype model of the Home Security System using ESP32-CAM, designed to simulate a real residential environment. The model features a miniature house setup with a clearly defined door and a mounting point for the camera module, representing the ESP32-CAM's position for monitoring the entry area. The design includes a visual boundary to indicate a secured perimeter and demonstrates where sensors and cameras would be placed in an actual home. This external view emphasizes the system's layout and gives a clear understanding of how the hardware components, such as the camera and sensors, are positioned for optimal detection shows how the automated security measures, including motion detection, live monitoring, and remote alerts, are applied in a controlled residential scenario.



Figure 7.2: Internal Circuit Connections and Component Integration of Home Security System

Prototype Model (Internal Circuit Connections):

The image provides a detailed look at the internal circuitry of the prototype, showing how all hardware components are integrated to form a functional home security system. At the core of the circuit is the Arduino Uno, which acts as the main controller and executes the logic for processing commands and controlling actuators. A breadboard is used to organize and connect the components, allowing for flexible wiring and testing. The servo motor is connected to the Arduino and serves as the mechanical actuator for opening and closing the door based on the commands received from the user. The HC-05 Bluetooth module is integrated into the circuit to enable wireless communication between the smartphone and Arduino, allowing remote control of the door and system operations. Jumper wires connect all components, ensuring proper power supply and signal transmission across modules. This internal view clearly demonstrates how the sensing, processing, communication, and actuation layers of the system are physically interconnected. It highlights the practical implementation of the IoT-enabled design, showing how motion detection, image capture, alert transmission, and remote control are coordinated within the prototype.

7.1 BLUETOOTH SCANNING



Figure 7.3 :Smartphone Interface Bluetooth Module Pairing

(Bluetooth Device Scanning Interface):

The first image shows the Bluetooth scanning interface on a smartphone used in the Home Security System using ESP32-CAM. It displays a list of nearby Bluetooth devices detected by the mobile application, including the HC-05 Bluetooth module connected to the Arduino Uno. This screen confirms that the system is ready to establish a wireless connection between the smartphone and the hardware unit. Identifying the HC-05 module ensures that the user can securely pair with the correct device before sending control commands. This step is essential for enabling remote control of the door mechanism after intrusion verification. The image shows the interface after selecting the indicating an active or in-progress connection. This screen represents successful communication setup between the smartphone and the Arduino Uno through Bluetooth.. This connection enables the user to remotely open or close the door, demonstrating reliable short-range wireless control within the home security system.

7.2 HC-05 MODULE CONNECTION INTERFACE

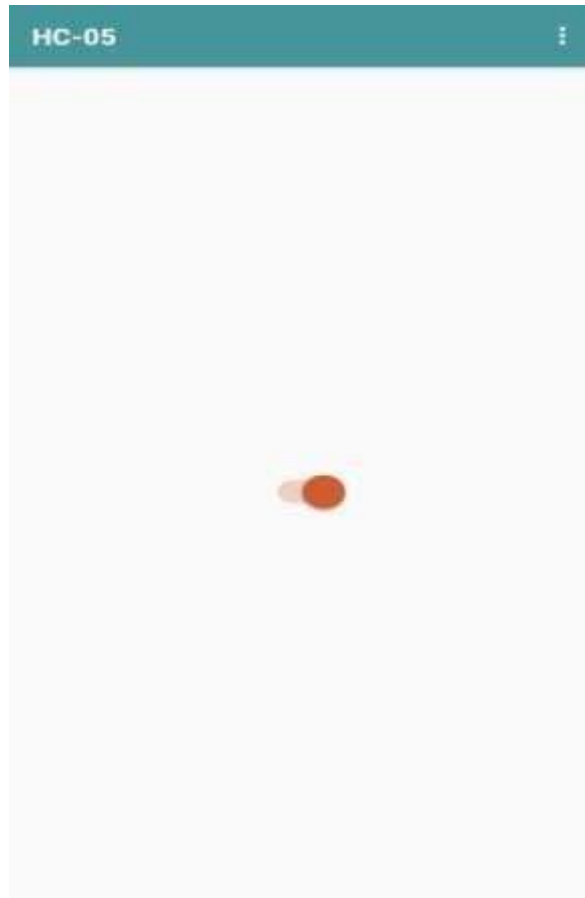


Figure 7.4 HC-05 Bluetooth Module Pair and Switch-Based Control

(HC-05 Bluetooth Connection Screen):

The image shows the interface after selecting the HC-05 Bluetooth module, indicating an active or in-progress connection. This screen represents successful communication setup between the smartphone and the Arduino Uno through Bluetooth.. This connection enables the user to remotely open or close the door, demonstrating reliable short-range wireless control within the home security system. The image shows the Bluetooth scanning interface on a smartphone used in the Home Security System using ESP32-CAM. It displays a list of nearby Bluetooth devices detected by the mobile application, including the HC-05 Bluetooth module connected to the Arduino Uno. This screen confirms that the system is ready to establish a wireless connection between the smartphone and the hardware unit. Identifying the HC-05 module ensures that the user can securely pair with the correct device before sending control commands. This step is essential for enabling remote control of the door mechanism after intrusion verification.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 TELEGRAM ALERT SHOWING REAL-TIME FACE DETECTION



Figure 8.1: Snapshots Telegram Alert Showing Real-Time Face Detection

The image illustrates a real-time security alert generated by the Home Security System using ESP32-CAM and transmitted to the user through a Telegram bot interface. When human presence is detected in the monitored area, the ESP32-CAM captures live images and processes them using a face/person detection algorithm. The detected individual is clearly marked with a green bounding box, visually indicating successful identification of a person within the frame. Along with the image, the system sends an alert message such as “Person detected” and displays a confidence score, and provide better visual confirmation to the user. This alert mechanism allows the homeowner to instantly verify the situation using a smartphone and take appropriate action, demonstrating the effectiveness of the system in providing intelligent surveillance, real-time intrusion detection, and prompt notification through IoT-enabled cloud communication.

8.2 SERVO MOTOR–BASED DOOR LOCK OPERATION USING ARDUINO BLUETOOTH CONTROL

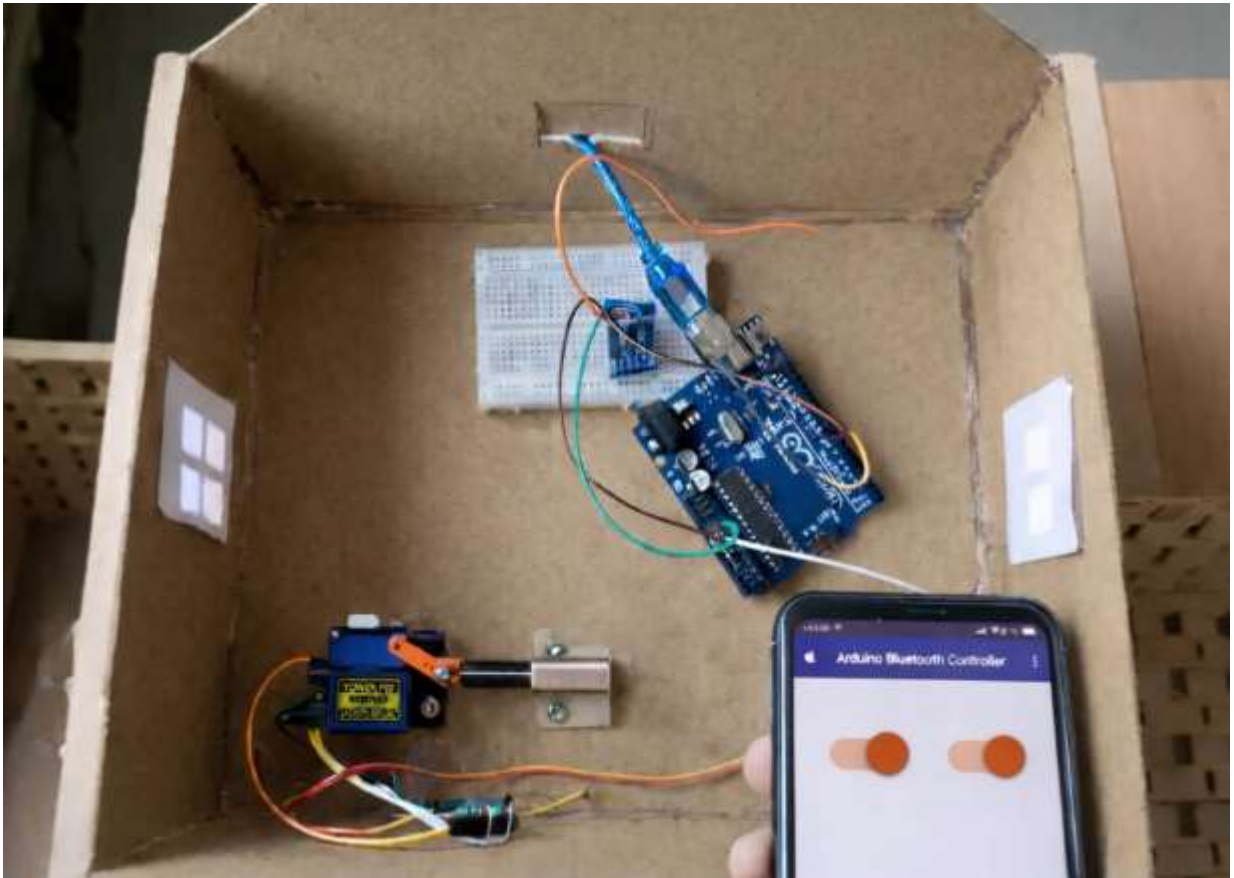


Figure 8.2: Servo Motor–Based Door Lock Operation Using Arduino Bluetooth Control

The above figure illustrates the working model of a smart door locking mechanism implemented as part of the ESP32-CAM based home security system. The setup is constructed inside a cardboard house prototype to demonstrate real-time operation in a residential environment.

An Arduino Uno acts as the main controller and is interfaced with an HC-05 Bluetooth module through a breadboard. The Bluetooth module enables wireless communication between the Arduino and a mobile device. A Tower Pro SG90 servo motor is mechanically connected to a sliding door latch mechanism, which functions as the door lock.

Using the Arduino Bluetooth Controller mobile application, the homeowner can remotely control the door lock through an orange toggle switch interface. When the switch is toggled ON, the Arduino receives the command via Bluetooth and rotates the servo motor to unlock the door. When the switch is toggled OFF, the servo motor rotates back to its original position, locking the door securely.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 CONCLUSION

This project presents the successful development of an intelligent home security and surveillance system using the ESP32-CAM module and IoT technologies. The system is designed to provide real-time monitoring by capturing images and serving them through an embedded web server, enabling users to remotely view live camera feeds using a standard web browser.

One of the key strengths of the proposed system is its ability to handle client requests continuously through the web server, ensuring uninterrupted real-time access to surveillance data. The ESP32-CAM's built-in Wi-Fi capability eliminates the need for additional networking hardware, making the system simple to deploy and suitable for small-scale residential environments. The use of embedded programming techniques enhances system reliability while maintaining low power consumption.

The project also demonstrates the practical integration of hardware and software components in an IoT-based application. Sensors, camera modules, and server-side handling work together seamlessly to detect activity and respond instantly to user requests. This integration highlights the effectiveness of embedded systems in delivering responsive and autonomous security solutions without relying on expensive commercial surveillance systems.

Furthermore, the developed system emphasizes scalability and adaptability. With minor modifications, additional features such as cloud storage, mobile application support, and AI-based image analysis can be incorporated. These enhancements can further improve detection accuracy, data accessibility, and overall system intelligence.

In addition, the project provides valuable learning outcomes in the areas of embedded systems, networking, and IoT application development. It enhances understanding of real-time data handling, server-client communication, and hardware-software co-design

9.2 FUTURE SCOPE

Although the proposed home security and surveillance system performs effectively, several enhancements can be implemented to improve its functionality, scalability, and intelligence. One potential improvement is the integration of advanced artificial intelligence techniques such as face recognition and object detection. By incorporating AI-based models, the system can distinguish between authorized users and intruders, thereby reducing false alerts and increasing overall security accuracy.

Another important area of future work is cloud integration. Storing captured images and event logs on a cloud platform would enable long-term data storage, remote access from anywhere, and better data management. A dedicated mobile application can also be developed to provide real-time notifications, live streaming, and system control, offering a more user-friendly and interactive experience compared to browser-based access.

The system can further be enhanced by improving hardware efficiency and scalability. Support for multiple camera modules and additional sensors such as smoke, gas, or door sensors can transform the system into a comprehensive smart home solution. Optimizing power consumption through energy-efficient modes or solar power integration would make the system more suitable for continuous operation.

In addition, future versions of the system can incorporate advanced networking features such as edge computing and local data processing. Processing images and events directly on the device can reduce network latency, minimize bandwidth usage, and ensure faster response times during critical situations. This approach also enhances reliability during network outages.

Finally, security and privacy aspects can be strengthened in future versions. Implementing encrypted communication, user authentication, and secure data transmission protocols will help protect sensitive surveillance data. These improvements will make the system more robust, reliable, and suitable for real-world deployment in residential and commercial environments.

REFERENCES

[1]. Design and Build a Home Security System Based on an ESP32-CAM

Philanthropy Y. A. Cahyono, Nugroho Suharto, Lis Diana Mustafa (2022).

ESP32-CAM captures intruder images and sends them via Telegram

[2]. Smart Wi-Fi Door Lock System Using ESP32-CAM and the Blynk App

J. P. Rohit, K. K. Naveen Kumar (2024).

Integrates ESP32-CAM and Blynk for IoT-based door locking. Enables camera monitoring and remote access control via smartphone.

[3]. Wi-Fi Door Lock System Using ESP32-CAM Based on IoT

S. Manikandan, M. Ramesh (2021).

Demonstrates door control via face recognition and IoT cloud. Highlights secure and user-friendly access management.

[4]. Home Door Security System with Face Recognition Using ESP32-CAM

S. S. Babu, R. K. Reddy et al. (2024).

Implements AI-based facial recognition for door access. ESP32-CAM sends alerts and captured images for remote verification.

[5]. Smart Home Warning System Using ESP32-CAM

P. Palkar, P. Bangar, S. Jadhav (2023).

ESP32-CAM detects motion and sends real-time alerts with images. Provides low-cost IoT surveillance without cloud dependency.

\

[6]. IOT-Based Smart Video Doorbell Using ESP32-CAM

M. Al-Shamiri, H. Al-Kindi, S. Ahmed (2023).

ESP32-CAM captures visitor images and sends instant notifications to homeowners via mobile applications.

[7]. Smart Garage Security System Using ESP32-CAM

R. Patel, N. Shah, K. Mehta (2021).

Detects unauthorized garage access using motion sensing and image capture with real-time alerts.

[8]. Vehicle Theft Detection System Using ESP32-CAM

S. Ramesh, V. Krishnan, P. Mohan (2022).

Monitors parked vehicles, captures suspicious activity images, and transmits alerts through IoT platforms.

[9]. Smart Baby Monitoring System Using ESP32-CAM

L. Chen, Y. Wang, Z. Liu (2023).

Provides live video streaming and motion-based alerts for infant monitoring via smartphone applications.

[10]. Office Entry Security System with Face Recognition

T. Rajesh, M. Senthil Kumar, K. Balaji (2024).

Uses ESP32-CAM facial authentication to control office access and maintain employee entry logs.

[11] Espressif Systems, ESP32-CAM

Development Board Datasheet, Espressif Systems Inc., 2023.

[12] R. Sanchez-Iborra, J. Santa,

“Low-Cost IoT Surveillance Systems Using Embedded Cameras,
” IEEE Internet of Things Journal, vol. 7, no. 5, pp. 4021–4030, 2020.

[13] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash,

“Internet of Things:
A Survey on Enabling Technologies, Protocols, and Applications,” IEEE Communications
Surveys & Tutorials, vol. 17, no. 4, pp. 2347–2376, 2015.

[14] S. Li, L. Da Xu, S. Zhao, “The Internet of Things: A Survey,”

Information Systems Frontiers, vol. 17, no. 2, pp. 243–259, 2015.

[15] A. Krizhevsky, I. Sutskever, G. E. Hinton,

“ImageNet Classification with Deep Convolutional Neural Networks,” Advances in Neural
Information Processing Systems (NIPS), 2012.

APPENDIX A

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ESP32-CAM	Espressif Systems 32-bit Wi-Fi Camera Module
PIR	Passive Infrared Sensor
CCTV	Closed-Circuit Television
IoT	Internet of Things
DFD	Data Flow Diagram
UML	Unified Modeling Language
VS Code	Visual Studio Code
OpenCV	Open Source Computer Vision Library
HOG	Histogram of Oriented Gradients
HTTP	HyperText Transfer Protocol
LED	Light Emitting Diode
PWM	Pulse Width Modulation
GUI	Graphical User Interface
VPN	Virtual Private Network
VS CODE	Visual Studio Code

APPENDIX B

1. Installation and Setup

```
# Clone repository
git clone <repository-url> cd
iot-home-security
```

2. Create Virtual Environment

```
python -m venv .venv source
.venv/bin/activate                # Linux / Mac
.venv\Scripts\activate            # Windows
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Configure ESP32-CAM and Telegram

```
# Set ESP32-CAM stream URL
export CAMERA_URL=http://192.168.43.60/cam-lo.jpg

# Set Telegram Bot credentials
export TELEGRAM_BOT_TOKEN=your_bot_token
export TELEGRAM_CHAT_ID=your_chat_id
```

5. Run Home Security Monitoring System

```
python p1.py
```

6. Stop the System

```
# Press 'q' in the display window or use Ctrl + C in terminal
```